

Login with Google (OAuth 2.0): step-by-step guide

For the E-Learning Platform (Spring Boot + Angular). Written for people who have never used Google Cloud Console or OAuth before. Follow the steps in order.

What you get: a "Continue with Google" button on the Login page. A new person can sign up with one click, and an existing customer can log in with Google. Your normal email + password login, your JWT and your refresh cookie all stay exactly as they are.

Contents

1. Read this first (what changes, what does not, honest status)
2. The idea in plain words (OAuth, Google ID token, how it fits your JWT)
3. Files you will touch (14 files, plus 1 optional)
4. STEP 1: Google Cloud Console (click by click)
5. STEP 2: Backend code
6. STEP 3: Frontend code
7. STEP 4: Run it and test it
8. If something goes wrong
9. Security rules for the team
10. What I tested and what I could not

1. Read this first

What this is. This is real OAuth 2.0 / OpenID Connect: Google proves who the person is, and your backend then gives that person your own JWT, the same one `/api/login` gives. Everything after login (every other endpoint, roles, the Angular interceptor, refresh cookie) is unchanged.

What does NOT change. - `/api/login`, `/api/register`, `/api/refresh`, `/api/logout` are untouched, so the SRS and the hidden Examly tests that use them are not affected. - The `User` table and the `User` model are untouched (no new columns). - `JwtUtil`, `JwtRequestFilter`, `RefreshTokenService`, `AuthController` are not edited. Google login is a new controller next to them.

What is new. One new endpoint `POST /api/auth/google`, three new Java files, one new Angular service, and a Google button on the login page.

Rules I chose (you can change them later): - A person who logs in with Google for the first time gets a **CUSTOMER** account. Nobody can become an **ADMIN** through Google. - An existing **ADMIN** account cannot log in with Google (admins keep using their password). - If someone already registered with the same email, Google login opens that same account (Google has verified the email). - A new Google user has no phone number, so the database gets the placeholder `0000000000`. The user can enter the real number on their profile page (it has a Mobile number field). - If `google.client-id` is empty, the Google button is hidden and the backend answers 400 to the endpoint. So the project still works with no Google setup at all.

Honest status (please read). - The **frontend** was built and tested in a real browser against a mock backend and a stand-in for Google's button: 14 checks passed, no errors. - The **java backend could not be compiled or run** in my environment (Maven downloads are blocked there). I checked the new files for syntax only. So the first time you start the backend, Maven must download one new library and compile the three new files. If you see a red error, copy it and send it to me. - The **real Google sign-in has not been run by me**, because it needs your own Google Cloud project. Step 4 shows how you test it.

Before you start, ask your SME one thing: "Is Google login (OAuth 2.0 / OpenID Connect) fine for the OAuth requirement, or do you specifically need Keycloak?" This guide is only for Google login.

2. The idea in plain words

Today (your JWT login). You type email and password. `AuthController.login` checks them in the database and `JwtUtil.generateToken` creates a JWT signed with your own secret (`jwt.secret`). Angular stores it and `AuthInterceptor` sends it as `Authorization: Bearer ...` on every request. `JwtRequestFilter` checks it.

OAuth 2.0 is a set of rules that lets one company (Google) prove who a person is to another app (yours), without your app ever seeing the person's Google password. **OpenID Connect (OIDC)** is the identity layer on top of OAuth 2.0. Google uses it, and it gives your app an **ID token**.

The four players | Name | In your project | |---| | User (resource owner) | the person at the login page | | Client | your Angular app | | Authorization server | Google (accounts.google.com) | | Resource server | your Spring Boot API |

The ID token is also a JWT, but it is signed by Google with Google's **private** key (RS256). Your own JWT is signed with a shared secret (HS256). Because Google publishes its **public** keys at a web address, your backend can check the signature without knowing any secret. Inside the token are claims such as `email`, `name`, `email_verified`, `aud` (which app it is for), `iss` (who issued it) and `exp` (when it expires).

The flow

1. User clicks "Continue with Google" on `/login`
2. Google's pop-up asks the user to pick an account
3. Google gives the browser an ID token ("`credential`")
4. Angular -> `POST /api/auth/google { "credential": "<ID token>" }`
5. Backend checks the ID token:
 - signature (Google's public keys)
 - iss = accounts.google.com
 - aud = YOUR client id (so a token made for another app is refused)

```

- exp = not expired
- email_verified = true
6. Backend finds the user by email, or creates a CUSTOMER
7. Backend answers exactly like /api/login:
  { token (your JWT), role, userId, username, email } + refresh cookie
8. Angular stores it like a normal login. Nothing else in the app changes.

```

The Client ID. Google gives your app a public "Client ID" (looks like 1234-abcd.apps.googleusercontent.com). It is not a secret. It tells Google which app is asking, and your backend uses it in check `aud`. We do **not** use a Client Secret in this design, so there is no secret to leak.

3. Files you will touch

#	File	Type	What to do
1	springapp/pom.xml	edit	add 1 dependency
2	springapp/src/main/resources/application.properties	edit	add 2 lines at the end
3	.../config/SecurityConfig.java	edit	add one path to one line
4	.../config/RateLimitFilter.java	optional edit	add one path
5	.../model/GoogleLoginDTO.java	NEW	copy whole file
6	.../service/GoogleAuthService.java	NEW	copy whole file
7	.../controller/GoogleAuthController.java	NEW	copy whole file
8	angularapp/src/environments/environment.ts	edit	add 1 line
9	angularapp/src/environments/environment.prod.ts	edit	add 1 line
10	angularapp/src/app/constant.ts	edit	add 1 line, change 1 line
11	angularapp/src/app/services/auth.service.ts	edit	add 1 method
12	angularapp/src/app/services/google-auth.service.ts	NEW	copy whole file
13	angularapp/src/app/components/login/login.component.ts	edit	replace whole file
14	angularapp/src/app/components/login/login.component.html	edit	replace whole file
15	angularapp/src/app/components/login/login.component.css	edit	add block at the end

(.../ means springapp/src/main/java/com/examly/springapp/.) Files 1-15 minus the optional one = 14 files.

Two people can split it: Person A does Step 1 (Google console) and the backend (files 1-7). Person B does the frontend (files 8-15). Person B needs the Client ID from Person A to test.

If you already applied CHANGES-7: these edits do not clash with it. `constant.ts` and `SecurityConfig.java` get only a small edit each, shown below as "find this, change to this".

4. STEP 1: Google Cloud Console (click by click)

Google changes the names of these screens now and then. If yours look a little different, look for the words in **bold**.

- Open **<https://console.cloud.google.com>** and sign in with a Gmail account (any one of the team can do this; use a normal Gmail, not a college or company account that may block it).
- At the top, click the **project picker** (it shows "Select a project"). Click **New project**. Name: `elearning-platform`. Click **Create**. Then make sure this project is selected in the picker.
- Open the left menu (three lines) -> **APIs & Services** -> **OAuth consent screen**. This opens the **Google Auth Platform** page. If it says "not configured yet", click **Get started**.
- App information:** App name `Online E-Learning Platform`; User support email: pick your Gmail. Click **Next**.
- Audience:** choose **External**. Click **Next**.
- Contact information:** type your email. Click **Next**.
- Tick the box to agree to the user data policy, click **Continue**, then **Create**.
- In the left menu of Google Auth Platform click **Clients**, then **+ Create client**.
- Application type:** choose **Web application**. Name: `E-Learning Web`.
- Under **Authorized JavaScript origins** click **+ Add URI** and add these (one by one):
 - `http://localhost:8081`
 - `http://localhost`
 - `http://localhost:4200` (only if you sometimes run Angular on 4200)

Google's own guide says to add **both** `http://localhost` and `http://localhost:<port>` for local testing. 11. **Authorized redirect URIs:** leave it **empty**. This button does not use redirects. 12. Click **Create**. A box appears with **Client ID** and **Client secret**. - **Copy the Client ID** (it ends with `.apps.googleusercontent.com`). Keep it in a notepad. - You do **not** need the Client secret. Do not paste it anywhere. 13. In the left menu click **Audience**. The status is **Testing**. Under **Test users** click **+ Add users** and add the Gmail address of **every person who will log in with Google** (all 7 of you, your SME, the evaluator). Click **Save**. - While the app is in Testing, only these listed Gmail accounts can sign in (up to 100). Anyone else gets "Access blocked". - You can click **Publish app** later to let anyone sign in. For the basic email/profile sign-in this does not need Google's review, but Google may show an "unverified app" notice. 14. Wait about 5 minutes. New settings can take a few minutes to start working.

When you deploy to a real website later: come back to **Clients** and add that website address (for example `https://yoursite.com`) under

5. STEP 2: Backend code

5.1 springapp/pom.xml: add one dependency

Inside `<dependencies> ... </dependencies>`, right after the `spring-boot-starter-security` block, add:

```
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-oauth2-jose</artifactId>
</dependency>
```

No version number is needed: Spring Boot already knows the right version. After saving, reload Maven (in IntelliJ: the small "Reload" icon in the Maven panel; in Eclipse: right click project -> Maven -> Update Project). This library is what checks Google's token signature.

5.2 application.properties: add at the very end

Do **not** replace the file. Only add these lines at the bottom:

```
# Login with Google: the public Client ID from Google Cloud Console (looks like 1234-abcd.apps.googleusercontent.com).
# Paste yours after the colon, or set the GOOGLE_CLIENT_ID environment variable. Leave empty to switch Google login off.
google.client-id=${GOOGLE_CLIENT_ID:}
```

Simplest for beginners: paste your Client ID after the colon, for example `google.client-id=${GOOGLE_CLIENT_ID:1234-abcd.apps.googleusercontent.com}`. A Client ID is a public identifier, not a password, so it is fine to keep it in the file. (Your database password is different: never commit that. Use the `DB_PASSWORD` environment variable.)

5.3 SecurityConfig.java: let the new endpoint through without a token

Open `springapp/src/main/java/com/examly/springapp/config/SecurityConfig.java`. Find this line:

```
.requestMatchers("/api/register", "/api/login", "/api/refresh", "/api/logout", "/api/health")
```

Change it to (only the new path is added):

```
.requestMatchers("/api/register", "/api/login", "/api/refresh", "/api/logout", "/api/health", "/api/auth/google")
```

Why: every other path needs a login token. This one is where people log in, so it must be open, just like `/api/login`. The line right after it (`.permitAll()`) stays as it is.

5.4 (Optional) RateLimitFilter.java: treat it like the other login calls

Your filter limits login attempts per minute. To include Google login, find this line (about line 38):

```
boolean authCall = path.equals("/api/login") || path.equals("/api/register") || path.equals("/api/refresh");
```

Change it to:

```
boolean authCall = path.equals("/api/login") || path.equals("/api/register") || path.equals("/api/refresh")
|| path.equals("/api/auth/google");
```

5.5 NEW file model/GoogleLoginDTO.java

Path: `springapp/src/main/java/com/examly/springapp/model/GoogleLoginDTO.java`

```
package com.examly.springapp.model;

import jakarta.validation.constraints.NotBlank;

/**
 * Request body of POST /api/auth/google. "credential" is the Google ID token that the
 * "Sign in with Google" button gives the browser.
 */
public class GoogleLoginDTO {
    @NotBlank(message = "Google credential is required")
    private String credential;

    /** No-arg constructor required by Jackson. */
    public GoogleLoginDTO() {}

    public String getCredential() { return credential; }
    public void setCredential(String credential) { this.credential = credential; }
}
```

5.6 NEW file service/GoogleAuthService.java

Path: `springapp/src/main/java/com/examly/springapp/service/GoogleAuthService.java`

What it does, in order: builds a decoder that downloads Google's public keys; checks the signature, expiry, issuer and that the token was made for **your** Client ID; requires a verified email; finds the user by email or creates a CUSTOMER; refuses ADMIN accounts.

```
package com.examly.springapp.service;

import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.DuplicateResourceException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.model.User;
import com.examly.springapp.repository.UserRepo;
import java.util.Locale;
import java.util.Set;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.core.DelegatingOAuth2TokenValidator;
import org.springframework.security.oauth2.core.OAuth2Error;
import org.springframework.security.oauth2.core.OAuth2TokenValidator;
import org.springframework.security.oauth2.core.OAuth2TokenValidatorResult;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.security.oauth2.jwt.JwtException;
import org.springframework.security.oauth2.jwt.JwtValidators;
import org.springframework.security.oauth2.jwt.NimbusJwtDecoder;
import org.springframework.stereotype.Service;

/**
 * "Login with Google" (OAuth 2.0 / OpenID Connect).
 * <p>
 * The browser gives us a Google ID token. This class proves the token is real by checking
 * Google's signature (public keys from Google), the issuer, the audience (our own client id)
 * and the expiry. Then it finds the user with that e-mail, or creates a CUSTOMER account.
 * The caller (GoogleAuthController) then issues the same JWT and refresh cookie as /api/login.
 * <p>
 * Nothing here is a secret: the client id is public. No password or Google token is logged.
 */
@Service
public class GoogleAuthService {
    private static final Logger log = LoggerFactory.getLogger(GoogleAuthService.class);
    private static final String GOOGLE_KEYS_URL = "https://www.googleapis.com/oauth2/v3/certs";
    private static final Set<String> GOOGLE_ISSUERS = Set.of("https://accounts.google.com", "accounts.google.com");
    /** The users table requires a 10 digit mobile number; Google does not give one. The user can change it later. */
    private static final String PLACEHOLDER_MOBILE = "0000000000";

    private final UserRepo userRepo;
    private final PasswordEncoder passwordEncoder;
    private final NimbusJwtDecoder decoder; // null when google.client-id is not set

    public GoogleAuthService(UserRepo userRepo, PasswordEncoder passwordEncoder,
        @Value("${google.client-id}") String clientId) {
        this.userRepo = userRepo;
        this.passwordEncoder = passwordEncoder;
        this.decoder = clientId == null || clientId.isBlank() ? null : buildDecoder(clientId.trim());
        if (this.decoder == null) {
            log.warn("google.client-id is not set: Login with Google is switched off");
        }
    }

    private static NimbusJwtDecoder buildDecoder(String clientId) {
        NimbusJwtDecoder d = NimbusJwtDecoder.withJwkSetUri(GOOGLE_KEYS_URL).build();
        OAuth2TokenValidator<Jwt> issuerCheck = jwt -> GOOGLE_ISSUERS.contains(jwt.getClaimAsString("iss"))
            ? OAuth2TokenValidatorResult.success()
            : OAuth2TokenValidatorResult.failure(new OAuth2Error("invalid_token", "Wrong token issuer", null));
        OAuth2TokenValidator<Jwt> audienceCheck = jwt -> jwt.getAudience() != null && jwt.getAudience().contains(clientId)
            ? OAuth2TokenValidatorResult.success()
            : OAuth2TokenValidatorResult.failure(new OAuth2Error("invalid_token", "Token is for another app", null));
        // createDefault() checks the expiry time; the other two checks are ours
        d.setJwtValidator(new DelegatingOAuth2TokenValidator<>(JwtValidators.createDefault(), issuerCheck, audienceCheck));
        return d;
    }

    /** Verifies the Google token and returns the matching (or newly created) user. */
    public User loginWithGoogle(String credential) {
        log.trace("loginWithGoogle entered");
        if (decoder == null) {
            throw new InvalidRequestException("Login with Google is not set up on this server");
        }
        Jwt jwt;
        try {
            jwt = decoder.decode(credential);
        } catch (JwtException e) {
            log.warn("Google login refused: token could not be verified");
            throw new BadCredentialsException("Invalid Google token");
        }
        String email = jwt.getClaimAsString("email");
        if (email == null || email.isBlank() || !Boolean.TRUE.equals(jwt.getClaimAsBoolean("email_verified"))) {
            log.warn("Google login refused: e-mail missing or not verified by Google");
            throw new BadCredentialsException("Google account e-mail is not verified");
        }
        String normalised = email.trim().toLowerCase(Locale.ROOT);

        User user = DatabaseOperationException.guard("loading a user", () -> userRepo.findByEmail(normalised)).orElse(null);
        if (user == null) {
            user = createCustomer(normalised, jwt.getClaimAsString("name"));
        }
        // Business rule: admin accounts keep using the password login only
        if (!"CUSTOMER".equals(user.getRole())) {
            log.warn("Google login refused for a non-customer account");
            throw new AccessDeniedException("Admin accounts must log in with their password");
        }
        log.info("User {} logged in with Google", user.getUserId());
        return user;
    }
}
```

```

private User createCustomer(String email, String googleName) {
    String name = googleName == null || googleName.isBlank() ? email.substring(0, email.indexOf('@')) : googleName.trim();
    if (name.length() > 50) {
        name = name.substring(0, 50);
    }
    // nobody knows this password, so the account can only be entered through Google (or "forgot password" later)
    User user = new User(email, passwordEncoder.encode(UUID.randomUUID().toString()), name, PLACEHOLDER_MOBILE, "CUSTOMER");
    try {
        User saved = userRepo.save(user);
        log.info("User {} created from Google login", saved.getId());
        return saved;
    } catch (DataIntegrityViolationException e) {
        // two Google logins for a new e-mail arrived at the same moment: use the one that won
        return userRepo.findByEmail(email)
            .orElseThrow(() -> new DuplicateResourceException("A user with this email already exists"));
    }
}
}

```

5.7 NEW file controller/GoogleAuthController.java

Path: springapp/src/main/java/com/examly/springapp/controller/GoogleAuthController.java

This builds the same answer as `/api/login` (your own JWT in `token`, plus the refresh cookie). It is a copy of the small `session(...)` logic in `AuthController`, so `AuthController` itself does not need to be edited.

```

package com.examly.springapp.controller;

import com.examly.springapp.config.JwtUtil;
import com.examly.springapp.model.GoogleLoginDTO;
import com.examly.springapp.model.User;
import com.examly.springapp.service.GoogleAuthService;
import com.examly.springapp.service.RefreshTokenService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import java.time.Duration;
import java.util.LinkedHashMap;
import java.util.Map;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseCookie;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

/**
 * POST /api/auth/google: exchanges a verified Google ID token for the same answer as /api/login
 * ({token, expiresIn, userId, role, username, email} plus the HttpOnly refresh cookie).
 */
@RestController
@RequestMapping("/api/auth")
@Tag(name = "Authentication", description = "Register, log in, refresh the access token and log out")
public class GoogleAuthController {
    private static final Logger log = LoggerFactory.getLogger(GoogleAuthController.class);
    private static final String REFRESH_COOKIE = "refresh_token";

    private final GoogleAuthService googleAuthService;
    private final JwtUtil jwtUtil;
    private final RefreshTokenService refreshTokenService;
    private final boolean secureCookie;
    private final long refreshDays;

    public GoogleAuthController(GoogleAuthService googleAuthService, JwtUtil jwtUtil,
                               RefreshTokenService refreshTokenService,
                               @Value("${app.cookie.secure:false}") boolean secureCookie,
                               @Value("${jwt.refresh-expiration-days:7}") long refreshDays) {
        this.googleAuthService = googleAuthService;
        this.jwtUtil = jwtUtil;
        this.refreshTokenService = refreshTokenService;
        this.secureCookie = secureCookie;
        this.refreshDays = refreshDays;
    }

    @Operation(summary = "Log in with Google",
               description = "Public. Body: {credential} = the Google ID token. Creates a CUSTOMER account on first use.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Logged in"),
        @ApiResponse(responseCode = "400", description = "Credential missing, or Google login is not configured"),
        @ApiResponse(responseCode = "401", description = "Google token is invalid, expired or the e-mail is not verified"),
        @ApiResponse(responseCode = "403", description = "Admin accounts cannot use Google login")
    })
    @PostMapping("/google")
    public ResponseEntity<Map<String, Object>> google(@Valid @RequestBody GoogleLoginDTO body) {
        log.trace("google login called");
        User user = googleAuthService.loginWithGoogle(body.getCredential());
        Map<String, Object> out = new LinkedHashMap<>();
        out.put("token", jwtUtil.generateToken(user.getEmail(), user.getRole(), user.getId(), user.getUsername()));
        out.put("expiresIn", jwtUtil.getAccessExpirationMs() / 1000);
        out.put("userId", user.getId());
        out.put("role", user.getRole());
        out.put("username", user.getUsername());
        out.put("email", user.getEmail());
        String refresh = refreshTokenService.issue(user);
        ResponseCookie cookie = ResponseCookie.from(REFRESH_COOKIE, refresh)
            .httpOnly(true)
            .secure(secureCookie)
            .sameSite("Strict")

```

```

        .path("/api")
        .maxAge(Duration.ofDays(refreshDays))
        .build();
    return ResponseEntity.status(HttpStatus.OK).header(HttpHeaders.SET_COOKIE, cookie.toString()).body(out);
}
}

```

6. STEP 3: Frontend code

6.1 environment.ts and environment.prod.ts: add the Client ID

In `angularapp/src/environments/environment.ts` add one line inside the object (do not forget the comma after the previous line):

```

export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080',
  aiSearchEnabled: true,
  // OAuth: the public "Client ID" from Google Cloud Console (not a secret). Leave '' to hide the Google button.
  googleClientId: '1234-abcd.apps.googleusercontent.com' // <-- paste YOUR Client ID here
};

```

Do the same in `environment.prod.ts` (add the line `googleClientId: ''` after `aiSearchEnabled: true,`).

If you leave it as `''`, the Google button simply does not appear and nothing else changes.

6.2 constant.ts: two small edits

Edit 1. Inside `export const API = { ... }` add one line (for example right after the `UPLOAD_IMAGE` line, or after `LOGOUT`):

```
GOOGLE_LOGIN: '/api/auth/google',
```

Edit 2. Find this line near the bottom:

```
export const PUBLIC_API_PATTERN = /\api\/(login|register|refresh|logout)$/;
```

Change it to (only `|auth\google` is added inside the brackets):

```
export const PUBLIC_API_PATTERN = /\api\/(login|register|refresh|logout|auth\google)$/;
```

Why: `AuthInterceptor` uses this list to know which calls are login calls (it must not attach an old token to them, and must not try to refresh on a 401 from them).

6.3 auth.service.ts: add ONE method

Open `angularapp/src/app/services/auth.service.ts`. Find the comment `/** Gets a new token using the refresh cookie.` and add this method **just above** it (inside the class):

```

/** OAuth: sends the Google ID token to the server and starts the same session as a normal login. */
loginWithGoogle(credential: string): Observable<any> {
  return this.http.post<any>(`${this.apiUrl}${API.GOOGLE_LOGIN}`, { credential }, { withCredentials: true }).pipe(
    tap((res: any) => this.applySession(res))
  );
}

```

Nothing else in this file changes. It reuses your existing `applySession`, so the token, role and user id are stored exactly like a normal login.

6.4 NEW file services/google-auth.service.ts

Path: `angularapp/src/app/services/google-auth.service.ts`. It loads Google's script and draws Google's button.

```

import { Injectable } from '@angular/core';
import { environment } from '../../../environments/environment';

/**
 * Loads Google's "Sign in with Google" script and draws its button.
 * The button gives us a Google ID token ("credential"); LoginComponent sends it to the backend.
 * When environment.googleClientId is empty nothing is loaded and the button stays hidden.
 */
@Injectable({
  providedIn: 'root'
})
export class GoogleAuthService {
  private static readonly SCRIPT_URL = 'https://accounts.google.com/gsi/client';
  private scriptLoaded: Promise<void> | null = null;

  /** True when a Google client id is configured. */
  get enabled(): boolean {
    return !!environment.googleClientId;
  }

  private loadScript(): Promise<void> {
    if (!this.scriptLoaded) {
      this.scriptLoaded = new Promise<void>((resolve, reject) => {
        const script = document.createElement('script');
        script.src = GoogleAuthService.SCRIPT_URL;
        script.async = true;

```

```

    script.defer = true;
    script.onload = () => resolve();
    script.onerror = () => {
      this.scriptLoaded = null; // allow a retry later
      reject(new Error('Could not load Google sign-in'));
    };
    document.head.appendChild(script);
  });
}
return this.scriptLoaded;
}

/** Draws the Google button inside "host"; onCredential gets the Google ID token when the user signs in. */
async renderButton(host: HTMLElement, onCredential: (credential: string) => void): Promise<void> {
  if (!this.enabled) {
    return;
  }
  await this.loadScript();
  const google = (window as any).google;
  google.accounts.id.initialize({
    client_id: environment.googleClientId,
    callback: (response: { credential: string }) => onCredential(response.credential)
  });
  google.accounts.id.renderButton(host, {
    type: 'standard',
    theme: 'filled_black',
    size: 'large',
    shape: 'pill',
    text: 'continue_with',
    width: Math.min(Math.max(host.clientWidth || 320, 200), 400)
  });
}
}
}

```

6.5 login.component.ts: replace the whole file

Path: angularapp/src/app/components/login/login.component.ts. Your existing `onSubmit()` is kept exactly as it was. Added: the Google imports, `googleEnabled`, `ngAfterViewInit`, `onGoogleCredential` and `goAfterLogin`.

```

import { AfterViewInit, Component, ElementRef, NgZone, ViewChild } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../services/auth.service';
import { CustomerService } from '../services/customer.service';
import { GoogleAuthService } from '../services/google-auth.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent implements AfterViewInit {
  loginForm: FormGroup;
  errorMessage = '';
  loading = false;
  showPassword = false;
  /** True when a Google client id is configured, so the Google button is shown. */
  googleEnabled = false;
  @ViewChild('googleButton') googleButton?: ElementRef<HTMLElement>;

  constructor(
    private fb: FormBuilder,
    private auth: AuthService,
    private customerService: CustomerService,
    private router: Router,
    private google: GoogleAuthService,
    private zone: NgZone
  ) {
    this.googleEnabled = this.google.enabled;
    this.loginForm = this.fb.group({
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required]]
    });
  }

  get email() { return this.loginForm.get('email'); }
  get password() { return this.loginForm.get('password'); }

  onSubmit(): void {
    if (this.loginForm.invalid || this.loading) { return; }
    this.loading = true;
    this.errorMessage = '';
    this.auth.login(this.loginForm.value).subscribe({
      next: (res: any) => {
        if (res.role === 'ADMIN') {
          this.loading = false;
          this.router.navigate(['/dashboard']);
          return;
        }
        // customers must have filled in their details once before they can use the app
        this.customerService.viewCustomerByUserId().subscribe({
          next: () => { this.loading = false; this.router.navigate(['/dashboard']); },
          error: (err) => {
            this.loading = false;
            if (err.status === 404) {
              this.router.navigate(['/customerdashboard']);
            } else {
              this.errorMessage = 'Could not load your profile. Please try again.';
            }
          }
        });
      },
      error: (err) => {
        this.loading = false;
        if (err.status === 401) {
          this.errorMessage = 'Invalid email or password';
        }
      }
    });
  }
}

```

```

    } else if (err.status === 429) {
      this.errorMessage = 'Too many attempts. Please wait a while and try again.';
    } else {
      this.errorMessage = 'Unable to reach the server. Please try again later.';
    }
  }
});
}

// ----- Login with Google (OAuth 2.0) -----

ngAfterViewInit(): void {
  if (this.googleEnabled && this.googleButton) {
    this.google.renderButton(this.googleButton.nativeElement, (credential) => this.onGoogleCredential(credential))
      .catch(() => { this.errorMessage = 'Google sign-in could not be loaded. Check your internet connection.'; });
  }
}

/** Google calls this outside Angular, so we step back into the Angular zone first. */
private onGoogleCredential(credential: string): void {
  this.zone.run(() => {
    if (this.loading) { return; }
    this.loading = true;
    this.errorMessage = '';
    this.auth.loginWithGoogle(credential).subscribe({
      next: (res: any) => this.goAfterLogin(res),
      error: (err) => {
        this.loading = false;
        if (err.status === 401) {
          this.errorMessage = 'Google sign-in failed. Please try again.';
        } else if (err.status === 403) {
          this.errorMessage = 'Admin accounts must log in with their email and password.';
        } else if (err.status === 400) {
          this.errorMessage = 'Login with Google is not set up on the server yet.';
        } else {
          this.errorMessage = 'Unable to reach the server. Please try again later.';
        }
      }
    });
  });
}

/** Same rule as the normal login: admins go to the dashboard, customers need their details saved once. */
private goAfterLogin(res: any): void {
  if (res.role === 'ADMIN') {
    this.loading = false;
    this.router.navigate(['/dashboard']);
    return;
  }
  this.customerService.viewCustomerById().subscribe({
    next: () => { this.loading = false; this.router.navigate(['/dashboard']); },
    error: (err) => {
      this.loading = false;
      if (err.status === 404) {
        this.router.navigate(['/customerdashboard']);
      } else {
        this.errorMessage = 'Could not load your profile. Please try again.';
      }
    }
  });
}
}

```

6.6 login.component.html: replace the whole file

Only the block marked with `*ngIf="googleEnabled"` is new.

```

<div class="auth">
  <section class="auth-hero">
    <h1>Learn at your own pace.</h1>
    <p>Browse courses, enroll in a few clicks and share what you thought of them. Pick up where you left off, on any device.</p>
    <ul>
      <li><mat-icon>menu_book</mat-icon>Courses from beginner lessons to certifications</li>
      <li><mat-icon>shopping_cart</mat-icon>Add to cart and enroll in one step</li>
      <li><mat-icon>star</mat-icon>Rate courses to help other learners</li>
    </ul>
  </section>

  <section class="auth-form">
    <div class="form-card">
      <h2>Login</h2>
      <p class="lead">Welcome back. Sign in to continue.</p>
      <form [formGroup]="loginForm" (ngSubmit)="onSubmit()" novalidate>
        <mat-form-field appearance="outline">
          <mat-label>Email</mat-label>
          <input matInput id="email" type="email" formControlName="email" autocomplete="username" />
          <mat-icon matSuffix>mail</mat-icon>
          <mat-error *ngIf="email?.errors?.['required']">Email is required</mat-error>
          <mat-error *ngIf="email?.errors?.['email']">Enter a valid email</mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline">
          <mat-label>Password</mat-label>
          <input matInput id="password" [type]="showPassword ? 'text' : 'password'" formControlName="password" autocomplete="current-password" />
          <button mat-icon-button matSuffix type="button" (click)="showPassword = !showPassword" [attr.aria-label]="showPassword ? 'Hide password' : 'Show password'">
            <mat-icon>{{ showPassword ? 'visibility_off' : 'visibility' }}</mat-icon>
          </button>
          <mat-error *ngIf="password?.errors?.['required']">Password is required</mat-error>
        </mat-form-field>

        <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>
        <button mat-flat-button color="primary" type="submit" id="loginButton" [disabled]="loginForm.invalid || loading">
          {{ loading ? 'Signing in...' : 'Login' }}
        </button>
      </form>
    </div>
  </section>

```

```

<ng-container *ngIf="googleEnabled">
  <div class="or"><span>or</span></div>
  <div #googleButton id="googleButton" class="google-btn"></div>
</ng-container>

<p class="alt">Don't have an account? <a routerLink="/registration">Register here</a></p>
</div>
</section>
</div>

```

6.7 login.component.css: add this at the very end

Keep everything that is already in the file. Paste this below it (it uses your Midnight colours):

```

/* Login with Google */
.or { display: flex; align-items: center; gap: 12px; margin: 20px 0 14px; color: var(--muted); font-size: 13px; }
.or::before, .or::after { content: ''; flex: 1; height: 1px; background: var(--line); }
.google-btn { display: flex; justify-content: center; min-height: 44px; }

```

7. STEP 4: Run it and test it

1. Make sure Step 1 is done, you copied the Client ID, and you pasted it into **both** `application.properties` (backend) and `environment.ts` (frontend). It must be the **same** Client ID in both places.
2. Start MySQL and the backend. In the console you must **not** see the warning `google.client-id is not set`. If Maven shows an error, send it to me.
3. Start the frontend with your usual command (`npm start`, port 8081).
4. Open <http://localhost:8081/login> in Chrome. Use the word `localhost`, not `127.0.0.1` (Google checks the exact address you registered).
5. You should see **Login**, the email/password form, a line "or", and a **Continue with Google** button.
6. Click it, pick a Gmail that you added as a **Test user**. First time: you land on the customer details page (same as a new registered customer). Fill it in once. After that you reach the dashboard.
7. Check the database. A new row should exist:

```
SELECT user_id, email, username, mobile_number, role FROM users ORDER BY user_id DESC LIMIT 3;
```

You should see your Gmail, your Google name, mobile `0000000000` and role `CUSTOMER`. 8. Log out and log in with Google again. It must open the **same** account (no second row). 9. Log in with an email + password account. It must still work exactly as before. 10. Try an admin: log in with the admin password. It still works. (An admin email used with Google is refused, on purpose.)

8. If something goes wrong

What you see	Why	Fix
No Google button at all	<code>googleClientId</code> is '' in <code>environment.ts</code> , or the page was not restarted	paste the Client ID, restart <code>npm start</code> , hard refresh (Ctrl+F5)
Google pop-up says Error 400: origin_mismatch	the address in the browser is not in "Authorized JavaScript origins"	add exactly <code>http://localhost:8081</code> and <code>http://localhost</code> in the console, wait 5 minutes; open the site as <code>localhost</code> , not <code>127.0.0.1</code>
Pop-up says Access blocked / app is being tested	that Gmail is not a Test user	Audience -> Test users -> add it
Red message "Google sign-in failed. Please try again."	backend refused the token (401): Client ID in backend differs from frontend, or token expired	use the same Client ID in both places; restart the backend; try again
"Admin accounts must log in with their email and password."	that email is an ADMIN account (403)	use the password form
"Login with Google is not set up on the server yet."	backend <code>google.client-id</code> is empty (400)	fill it in <code>application.properties</code> , restart the backend
"Google sign-in could not be loaded"	no internet, or a blocker stops <code>accounts.google.com</code>	check the network, disable the blocker for this site
Backend does not start: <code>package org.springframework.security.oauth2...</code> does not exist	the pom dependency was not added or Maven did not reload	add the dependency from 5.1 and reload Maven
Backend log: Google login refused: token could not be verified	wrong Client ID, expired token, or the backend cannot reach <code>www.googleapis.com</code>	check the Client ID; check the server has internet
After Google login you see a 404 on the customer page	normal for a new user: they must save their details once	fill the customer details form

9. Security rules for the team

- **Never commit** your database password, your JWT secret or any Google **Client secret**. This design needs no Client secret at all, so if you ever see one, leave it out of the code.
- The **Client ID** is public. It is fine to keep it in the code.
- The backend checks the token itself (signature, issuer, audience, expiry, verified email). Never trust an email sent by the browser without a

- Google token.
 - Admin accounts cannot be reached through Google login.
 - Rotate any password or key that was ever pushed to GitHub (the repo's `application.properties` still has a real DB password default: change it and use `DB_PASSWORD`).
 - Known limit: users are matched by email (your `users` table has no Google id column). That is fine for this project; a bigger system would store Google's `sub` id as well.
-

10. What I tested and what I could not

Tested (real browser, mock backend, stand-in Google button): 14 checks, 0 failed, no page errors - Google script is requested once; the button is drawn inside `#googleButton` with the dark theme and "continue with" text. - The normal email/password form is still there and an admin password login still works. - A bad token shows "Google sign-in failed" and stays on the login page. - An admin account shows "Admin accounts must log in with their email and password." - A good token stores the token, role and session flag like a normal login and goes to the customer details page. - With an empty Client ID, the Google block is not shown and Google's script is never requested. - Mobile layout shows the button. `ng serve` compiles with no type errors.

Not tested by me - The Java backend (never compiled or started here; syntax-checked only). The library version comes from Spring Boot 3.2.5, so it should resolve, but please run it and tell me any red error. - The real Google pop-up with a real account. - Your existing unit tests (`ng test`) were not re-run for this change. - Examy's hidden tests: your old endpoints are untouched. If tests fail around the login page, set `googleClientId: ''` in `environment.ts`.